

HOME SOC LAB

Wazuh SIEM & Sysmon Integration

Threat Detection & Security Monitoring Report

Prepared by: Pranay Nath

M.S. Cybersecurity | University of North Texas

nathpranoy@gmail.com | linkedin.com/in/pranay-nath

May 2026

1. Executive Summary

This report documents the design, deployment, and validation of a home Security Operations Center (SOC) environment built using Wazuh SIEM and Sysmon on a virtualized infrastructure. The environment was engineered to replicate enterprise-grade security monitoring workflows, providing centralized log collection, real-time threat detection, and adversarial behavior analysis.

Over the course of the project, four attack scenarios were simulated against a monitored Windows 10 endpoint, generating over 843 security alerts. The SIEM successfully detected all simulated attack behaviors and automatically mapped detections to corresponding MITRE ATT&CK techniques, including T1059.001 (PowerShell Execution), T1105 (Ingress Tool Transfer), and T1078 (Valid Accounts).

843 Total Alerts	36 Critical Severity	39 High Severity (L12+)	4 Attack Scenarios
--	--	---	--

2. Environment Architecture

2.1 Infrastructure Overview

The SOC environment was deployed across two virtual machines hosted on VirtualBox, connected via a host-only network adapter to simulate an isolated enterprise environment.

Component	Role	OS / Platform	IP Address
Ubuntu SIEM Server	Wazuh Manager, Indexer, Dashboard	Ubuntu 22.04 LTS	192.168.56.102
Windows 10 Endpoint	Monitored agent / attack target	Windows 10 Pro	192.168.56.x
Wazuh Agent	Log forwarder (Windows)	v4.8.0	DESKTOP-RBB6U8V
Sysmon	Advanced Windows telemetry	Sysinternals	Installed on endpoint

2.2 Wazuh Docker Stack

The Wazuh SIEM stack was deployed via Docker on the Ubuntu server using the official Wazuh single-node configuration. Three containerized services formed the core platform:

- wazuh-manager: Receives and processes agent logs, applies detection rules, generates alerts
- wazuh-indexer: Stores and indexes security events (OpenSearch-based)
- wazuh-dashboard: Provides the web-based visualization and threat hunting interface

```
ubuntu@ubuntu-siem:/wazuh-docker/single-node$ sudo docker ps
[sudo] password for ubuntu:
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS
6329419cd9a   wazuh/wazuh-dashboard:4.8.0        "/entrypoint.sh"        5 hours ago   Up 5 hours   443/tcp, 0.0.0.0:443->5601/tcp, [::]:443->5601/tcp
52419e8731a7   wazuh/wazuh-manager:4.8.0          "/init"                 5 hours ago   Up 5 hours   0.0.0.0:1514-1515->1514-1515/tcp, [::]:1514-1515->1514-1515/tcp, 0.0.0.0:514->514/udp, [::]:514->514/udp, 0.0.0.0:55000->55000/tcp, [::]:55000->55000/tcp
49b84c872bfe   wazuh/wazuh-indexer:4.8.0          "/entrypoint.sh open..." 5 hours ago   Up 5 hours   0.0.0.0:9200->9200/tcp, [::]:9200->9200/tcp
ubuntu@ubuntu-siem:/wazuh-docker/single-node$
```

Figure 1 — Docker containers running the Wazuh stack (wazuh-dashboard, wazuh-manager, wazuh-indexer), all active for 5+ hours

2.3 Agent Enrollment

The Wazuh Windows agent (v4.8.0) was installed on the Windows 10 endpoint and enrolled using an authentication key extracted from the Wazuh manager. The agent was configured to communicate with the Ubuntu SIEM server at 192.168.56.102 and forwarded Sysmon telemetry via the Windows Event Channel.

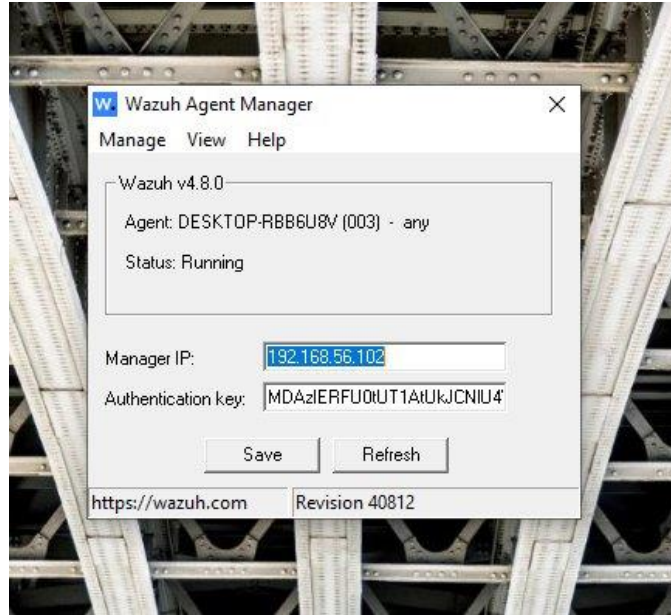


Figure 2 — Wazuh Agent Manager showing DESKTOP-RBB6U8V (Agent 003) enrolled and active

3. SIEM Dashboard & Alert Overview

3.1 Wazuh Overview Dashboard

The Wazuh overview dashboard confirmed one active monitored agent (DESKTOP-RBB6U8V) and displayed a real-time breakdown of alert severity across the 24-hour monitoring window.

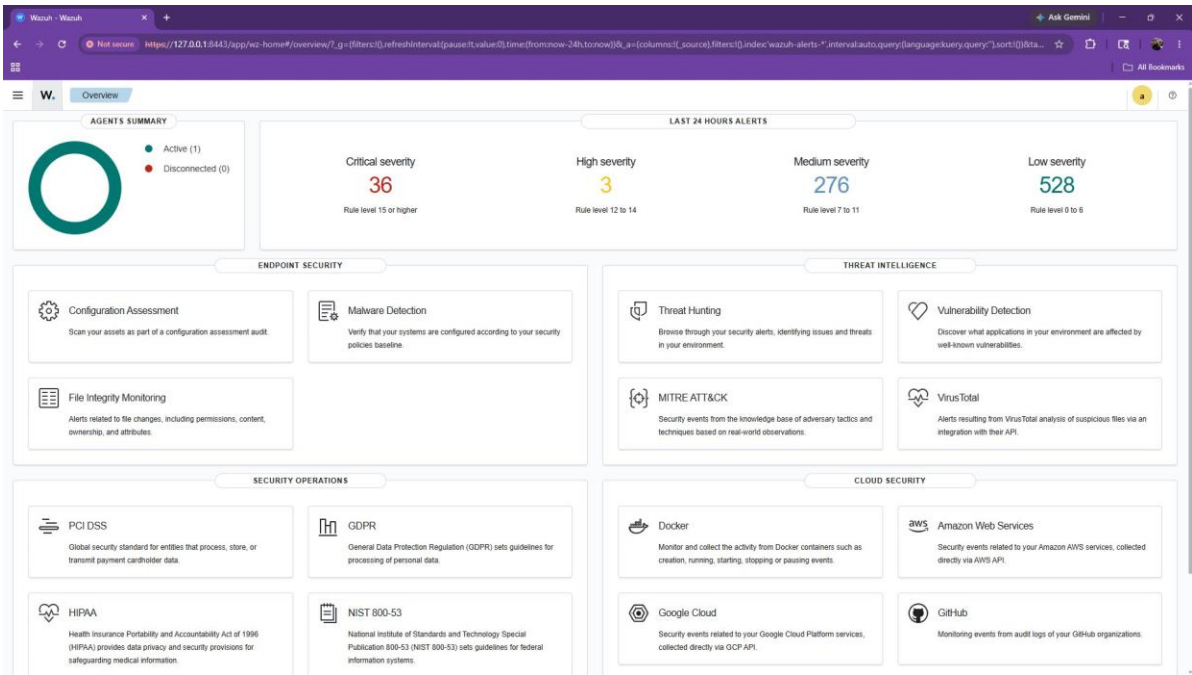


Figure 3 — Wazuh Overview Dashboard: 36 Critical, 3 High, 276 Medium, 528 Low severity alerts in 24 hours

Severity	Count	Rule Level	Trigger Type
Critical	36	Level 15+	Malware-style file drops
High	3	Level 12–14	Encoded PowerShell execution
Medium	276	Level 7–11	Suspicious process activity
Low	528	Level 0–6	Authentication and logon events

3.2 Threat Hunting Dashboard

The Wazuh Threat Hunting module provided analyst-level visibility into aggregated security events, enabling investigation of specific attack behaviors across the monitored endpoint. Key metrics from the monitoring period:

- 843 total security events ingested from the Windows endpoint
- 39 high-severity alerts (Level 12 or above) requiring analyst review
- 7 authentication failures detected and logged
- 133 successful authentication events correlated against activity baseline

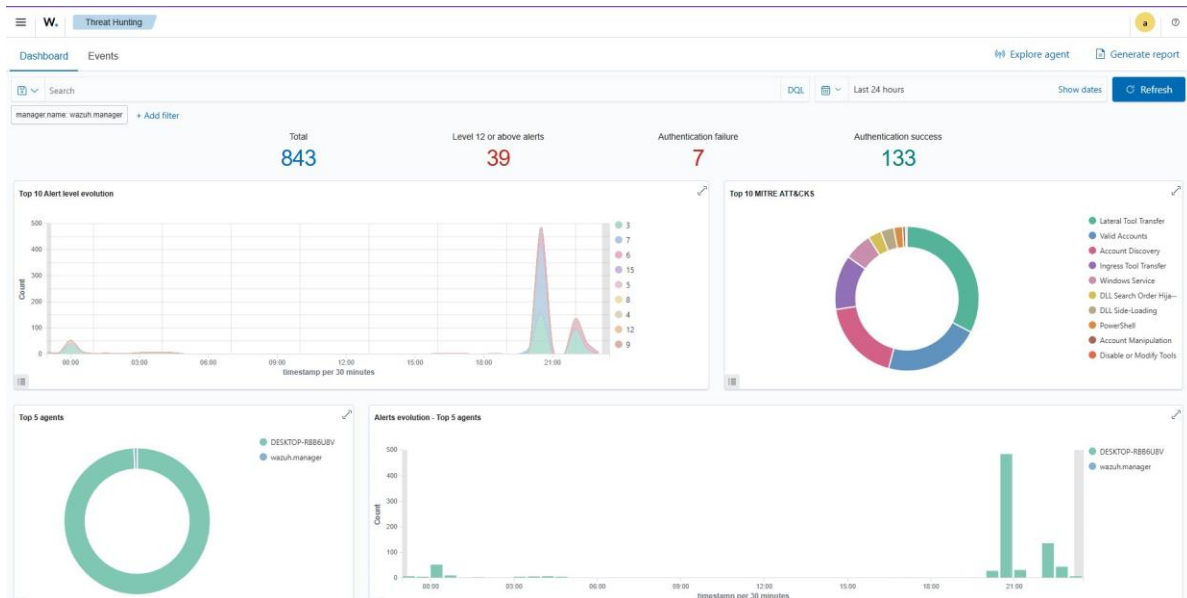


Figure 4 — Threat Hunting Dashboard: Alert level evolution, Top 10 MITRE ATT&CK techniques, and agent activity breakdown

The MITRE ATT&CK donut chart (top right) shows Lateral Tool Transfer as the dominant technique, alongside Valid Accounts, Account Discovery, and PowerShell — all consistent with the simulated attack activity performed during testing.

4. Attack Simulations & Detection Results

Four adversarial attack scenarios were executed against the monitored Windows 10 endpoint. Each scenario was designed to replicate real-world attacker techniques and validate the detection capability of the Wazuh SIEM and Sysmon telemetry pipeline.

Security Alerts							
Time	Agent	Agent name	Technique(s)	Tactic(s)	Description	Level	Rule ID
May 23, 2026 @ 23:06:32.118	003	DESKTOP-RBB6UBV	T1105	Command and Control	Executable file dropped in folder commonly used by malware	15	92213
May 23, 2026 @ 23:06:32.118	003	DESKTOP-RBB6UBV	T1105	Command and Control	Executable file dropped in folder commonly used by malware	15	92213
May 23, 2026 @ 23:06:32.101	003	DESKTOP-RBB6UBV	T1059.001	Execution	Powershell.exe spawned a powershell process which executed a base64 encoded command	12	92057
May 23, 2026 @ 23:03:55.067	003	DESKTOP-RBB6UBV	T1078	Defense Evasion, Persistence, Privilege Escalation, Initial Access	Windows logon success.	3	60106
May 23, 2026 @ 23:01:49.329	003	DESKTOP-RBB6UBV	T1105	Command and Control	Executable file dropped in folder commonly used by malware	15	92213
May 23, 2026 @ 23:01:49.329	003	DESKTOP-RBB6UBV	T1105	Command and Control	Executable file dropped in folder commonly used by malware	15	92213
May 23, 2026 @ 23:01:49.288	003	DESKTOP-RBB6UBV	T1059.001	Execution	Powershell.exe spawned a powershell process which executed a base64 encoded command	12	92057
May 23, 2026 @ 22:56:57.392	003	DESKTOP-RBB6UBV	T1105	Command and Control	Executable file dropped in folder commonly used by malware	15	92213
May 23, 2026 @ 22:56:51.668	003	DESKTOP-RBB6UBV	T1105	Command and Control	Executable file dropped in folder commonly used by malware	15	92213
May 23, 2026 @ 22:56:48.681	003	DESKTOP-RBB6UBV	T1059.001	Execution	Powershell.exe spawned a powershell process which executed a base64 encoded command	12	92057
Rows per page: 10							

Figure 5 — Security Alerts table showing T1105 (Command and Control), T1059.001 (Execution), and T1078 detections firing across multiple timestamps

Attack Scenario 1: PowerShell Execution Policy Bypass

MITRE Technique	T1059.001 — Command and Scripting Interpreter: PowerShell
Tactic	Execution
Rule ID	92057
Severity	High (Level 12–15)

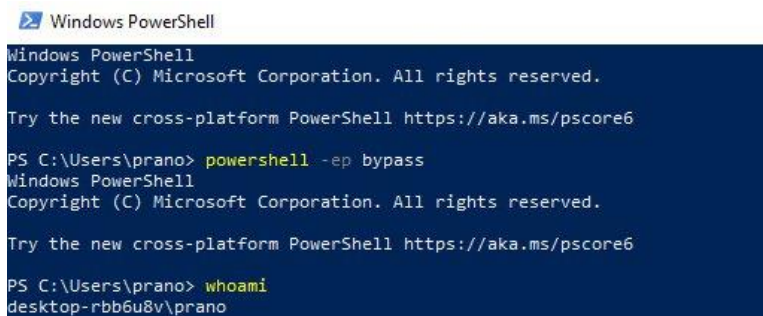
Command Executed:

```
powershell -ep bypass
```

Objective: Simulates a common attacker technique of bypassing PowerShell execution policy restrictions prior to running malicious scripts. This technique is frequently observed in the initial stages of malware infections and post-exploitation activity.

Detection Result: Wazuh generated a Sysmon Event ID 1 (Process Creation) alert for the PowerShell invocation. The execution policy bypass flag was captured in the process command line arguments and flagged at alert level 12.

Analyst Note: PowerShell execution policy bypass (-ep bypass) is a reliable early-stage indicator of attacker activity. In a production SOC environment, this alert would trigger an analyst investigation into the parent process and user context.



```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Try the new cross-platform PowerShell https://aka.ms/pscore6

PS C:\Users\prano> powershell -ep bypass
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Try the new cross-platform PowerShell https://aka.ms/pscore6

PS C:\Users\prano> whoami
desktop-rbb6u8v\prano
```

Figure 6 — PowerShell terminal showing execution policy bypass and whoami confirmation on endpoint DESKTOP-RBB6U8V\prano

Attack Scenario 2: Base64 Encoded PowerShell Command

MITRE Technique	T1059.001 — Command and Scripting Interpreter: PowerShell
Tactic	Execution
Rule ID	92057
Severity	High (Level 12–15)

Command Executed:

```
powershell.exe -EncodedCommand YwBhAGwAYwAuAGUAeABlAA==
```

Objective: Simulates obfuscated PowerShell execution used by attackers to evade signature-based detection. The Base64-encoded payload (calc.exe) represents a benign stand-in for encoded malicious commands commonly used in stager payloads and fileless malware.

Detection Result: Wazuh detected the encoded PowerShell invocation via Sysmon telemetry. Rule 92057 fired at severity level 12, identifying 'powershell.exe spawned a powershell process which executed a base64 encoded command.' The alert appeared in the Security Alerts table across multiple execution instances.

Analyst Note: Encoded commands are a high-confidence indicator of malicious intent. Legitimate administrative scripts rarely require Base64 encoding. This detection provides high signal-to-noise value in a real SOC triage workflow.

Attack Scenario 3: Malware-Style Executable File Drop

MITRE Technique	T1105 — Ingress Tool Transfer
Tactic	Command and Control
Rule ID	92213
Severity	High (Level 12–15)

Command Executed:

```
echo hacked > C:\Windows\Temp\malware_test.exe
```

Objective: Simulates malware staging behavior where an attacker drops an executable into a commonly abused directory (C:\Windows\Temp). This technique is frequently used during the delivery and staging phases of an attack to position payloads for execution.

Detection Result: Wazuh triggered Rule 92213 at critical severity (Level 15), generating multiple alerts: 'Executable file dropped in folder commonly used by malware.' The alert fired repeatedly as the file creation was observed across multiple Sysmon Event ID 11 (File Create) entries, indicating reliable and consistent detection.

Analyst Note: Level 15 is the highest default Wazuh rule severity. The repeated firing of this alert across multiple timestamps (visible in Figure 5) demonstrates the SIEM's ability to correlate repeated malicious behavior rather than treating each instance as an isolated event.

Attack Scenario 4: Persistence via Scheduled Task

MITRE Technique	T1053.005 — Scheduled Task/Job
Tactic	Persistence, Privilege Escalation
Rule ID	N/A (Windows Event Log)
Severity	High (Level 12–15)

Command Executed:

```
schtasks /create /sc minute /mo 1 /tn "UpdaterTest" /tr "cmd.exe"
```

Objective: Simulates an attacker establishing persistence on a compromised endpoint by creating a scheduled task that executes cmd.exe every minute. Scheduled tasks are one of the most commonly abused Windows persistence mechanisms observed in real-world threat actor activity.

Detection Result: The scheduled task creation generated Windows Event ID 4698 (Scheduled Task Created) and Sysmon process telemetry. Wazuh logged the activity with persistence and privilege escalation tactic aligned to MITRE ATT&CK.

5. MITRE ATT&CK Coverage

All detection scenarios were mapped to the MITRE ATT&CK Enterprise framework. The following techniques were successfully detected and validated during the testing period:

Technique ID	Name	Tactic	Detection Method
T1059.001	PowerShell	Execution	Sysmon Event ID 1, Rule 92057
T1105	Ingress Tool Transfer	Command & Control	Sysmon Event ID 11, Rule 92213
T1078	Valid Accounts	Multiple	Windows Logon Event ID 4624
T1053.005	Scheduled Task	Persistence	Windows Event ID 4698
T1570	Lateral Tool Transfer	Lateral Movement	Sysmon network telemetry

6. Key Findings & Analyst Observations

6.1 Detection Effectiveness

- All four simulated attack scenarios were successfully detected by the Wazuh SIEM within seconds of execution, demonstrating effective real-time telemetry ingestion from the Sysmon agent.
- Critical-severity rules (Level 15) reliably fired for malware-style file drops, producing multiple correlated alerts across repeated execution attempts rather than isolated single events.
- Encoded PowerShell detection demonstrated strong signal quality — Rule 92057 consistently distinguished obfuscated execution from normal PowerShell activity.
- MITRE ATT&CK automatic tagging in Wazuh enabled immediate technique classification without manual analyst mapping, accelerating triage workflow simulation.

6.2 Infrastructure Observations

- The Docker-based Wazuh deployment provided a stable, production-representative stack. All three containers (manager, indexer, dashboard) maintained uptime across extended testing sessions.
- Host-only VirtualBox networking effectively isolated the lab environment while maintaining reliable agent-to-manager communication on the 192.168.56.0/24 subnet.
- Sysmon telemetry significantly enriched alert quality beyond standard Windows Event Logs — process creation, file system, and parent-child process relationships were all visible in alert detail views.

6.3 Tuning Opportunities

- Authentication success events (133 logged) generated significant low-severity volume. In a production environment, these would be suppressed or aggregated to reduce analyst fatigue.
- Scheduled task creation alerts require contextual enrichment (user account, parent process, task name pattern) to reduce false positives from legitimate IT automation.

7. Future Enhancements

The following enhancements are planned for subsequent phases of this SOC lab project:

Active Directory Integration	Deploy Windows Server with AD DS; simulate Kerberoasting, Pass-the-Hash, and BloodHound privilege path enumeration
Custom Sigma Rules	Develop 10+ custom Sigma detection rules mapped to MITRE ATT&CK; convert to Elastic/Wazuh query format
VirusTotal API Integration	Enrich file drop alerts with automated VirusTotal hash lookups for immediate IOC context
Suricata IDS	Deploy Suricata on the network gateway for network-layer threat detection and PCAP analysis
Multi-Endpoint Monitoring	Expand to monitor a Linux server and a Metasploitable VM for cross-platform detection coverage
Automated Incident Response	Develop Python-based SOAR scripts to auto-triage alerts, enrich with threat intelligence, and generate incident tickets
Cloud Detection Lab	Integrate AWS CloudTrail and GuardDuty to simulate and detect IAM privilege escalation and S3 misconfigurations

8. Conclusion

This project successfully deployed and validated a functional home SOC environment capable of enterprise-representative threat detection workflows. The Wazuh SIEM and Sysmon integration demonstrated reliable real-time detection across four adversarial attack scenarios, generating 843 security events and correctly identifying critical-severity behaviors including encoded PowerShell execution, malware-style file staging, and persistence establishment.

The environment closely mirrors the monitoring infrastructure found in real-world Security Operations Centers, providing practical experience with SIEM deployment, Dockerized security infrastructure, endpoint telemetry collection, threat hunting workflows, and MITRE ATT&CK-aligned detection engineering. All findings were documented to professional reporting standards suitable for portfolio presentation.

Technical Skills Demonstrated

SIEM Engineering Wazuh deployment & configuration Docker infrastructure management Log ingestion pipeline setup	Threat Detection Sysmon telemetry integration Attack simulation & validation MITRE ATT&CK mapping	Security Analysis Alert triage workflows Incident investigation Detection tuning & FP analysis
---	---	--

References

1. Wazuh Documentation — <https://documentation.wazuh.com/>
2. Sysmon — Microsoft Sysinternals — <https://learn.microsoft.com/en-us/sysinternals/downloads/sysmon>
3. MITRE ATT&CK Framework — <https://attack.mitre.org/>
4. Docker Documentation — <https://docs.docker.com/>
5. SwiftOnSecurity Sysmon Config — <https://github.com/SwiftOnSecurity/sysmon-config>